

CSS Fundamentals

Course Information

Course Name: CSS Fundamentals

Level: Beginner

Prerequisites: Basic knowledge of HTML

Estimated Duration: 8–10 Hours

Course Description

CSS (Cascading Style Sheets) is used to control the appearance and layout of HTML webpages. In this course, students will learn how to style text, colors, backgrounds, borders, spacing, and layouts. Students will also learn Flexbox, Grid, responsive design, positioning, pseudo-classes, and basic CSS animations. By the end of the course, students will be able to create attractive and responsive webpages.

Learning Outcomes

After completing this course, students will be able to:

- Explain what CSS is and why it is used.
 - Apply CSS to HTML documents.
 - Use selectors and CSS properties.
 - Change colors, fonts, and text styles.
 - Understand the CSS box model.
 - Control margins, padding, and borders.
 - Create layouts using Flexbox.
 - Create layouts using CSS Grid.
 - Position elements on a webpage.
 - Create responsive webpages.
 - Use pseudo-classes and pseudo-elements.
 - Create basic transitions and animations.
 - Build a styled responsive webpage.
-

Lesson 1: Introduction to CSS

Learning Objectives

After this lesson, students will be able to:

- Define CSS.
- Explain the purpose of CSS.
- Understand the relationship between HTML and CSS.
- Apply basic CSS styling.

What is CSS?

CSS stands for **Cascading Style Sheets**.

CSS is used to control the appearance and layout of HTML elements.

HTML provides the **structure** of a webpage, while CSS provides its **appearance and design**.

For example:

HTML:

```
```html id="9b5w0a" <h1>Hello World</h1>
```

CSS:

```
```css id="4gdw6x"
h1 {
  color: blue;
}
```

The CSS changes the color of the heading.

Lesson 2: Ways to Add CSS

There are three common ways to apply CSS.

1. Inline CSS

CSS can be written directly inside an HTML element.

```
```html id="t1z5k6" <h1 style="color: blue;"> Hello World </h1>
```

Inline CSS is useful for small changes but is generally not recommended for large websites.

---

## ## 2. Internal CSS

CSS can be written inside the ``<style>`` element.

```
```html id="v8u6e2"
<!DOCTYPE html>
<html>

<head>

<style>

h1 {
  color: blue;
}

</style>

</head>

<body>

<h1>Hello World</h1>

</body>

</html>
```

3. External CSS

CSS can be stored in a separate file.

HTML:

```
```html id="9a1ksp" <link rel="stylesheet" href="style.css">
```

CSS:

```
```css id="8t7n6j"
h1 {
```

```
    color: blue;
}
```

External CSS is the preferred approach for most websites because it keeps structure and styling separate.

Lesson 3: CSS Syntax

A CSS rule normally contains a selector, properties, and values.

```
``css id="m6j5q8" p { color: red; font-size: 20px; }
```

The parts are:

- `p` → selector
- `color` → property
- `red` → value
- `font-size` → property
- `20px` → value

Lesson 4: CSS Selectors

Selectors determine which HTML elements should be styled.

Element Selector

```
``css id="2az5j8"
p {
  color: green;
}
```

This applies to all `<p>` elements.

Class Selector

HTML:

```
``html id="9w7r2p" <p class="highlight"> Important information </p>
```

CSS:

```
``css id="4s8x1c"
```

```
.highlight {  
  color: red;  
}
```

Classes can be reused on multiple elements.

ID Selector

HTML:

```
<html id="d6j8y3" <h1 id="title"> My Website </h1>
```

CSS:

```
css id="f4h9c1"  
#title {  
  color: blue;  
}
```

An ID should normally identify one unique element on a page.

Universal Selector

```
css id="6m0w1a" * { margin: 0; padding: 0; }
```

The universal selector selects all elements.

Grouping Selectors

Multiple selectors can be grouped.

```
css id="j7e3q4"  
h1, h2, p {  
  font-family: Arial;  
}
```

Lesson 5: Colors

CSS provides several ways to specify colors.

Color Name

```
``css id="m2y8k7" h1 { color: red; }
```

HEX

```
``css id="6w3f1r"
h1 {
  color: #ff0000;
}
```

RGB

```
``css id="v3k8p2" h1 { color: rgb(255, 0, 0); }
```

RGBA

RGBA includes transparency.

```
``css id="7x5d9m"
div {
  background-color: rgba(0, 0, 255, 0.5);
}
```

The fourth value represents opacity.

Lesson 6: Backgrounds

CSS can control the background of elements.

Background Color

```
``css id="4c8z2m" body { background-color: lightgray; }
```

Background Image

```
``css id="n9p2q7"
```

```
body {  
    background-image: url("background.jpg");  
}
```

Background Size

```
```css id="e3w7k1" body { background-size: cover; }
```

```

Background Position

```css id="a8d4f2"  
body {  
    background-position: center;  
}
```

Lesson 7: Text and Fonts

CSS provides many properties for styling text.

Text Color

```
```css id="r5v7x2" p { color: #333; }
```

```

Font Size

```css id="6k9m3s"  
p {  
    font-size: 18px;  
}
```

Font Family

```
```css id="q8z2m5" body { font-family: Arial, sans-serif; }
```

```

Font Weight
```

```
``css id="y4n7c8"
h1 {
 font-weight: bold;
}
```

---

## Text Alignment

```
``css id="h2f6k9" h1 { text-align: center; }
```

Possible values include:

- left
- center
- right
- justify

---

### ## Text Decoration

```
``css id="s7j3p1"
a {
 text-decoration: none;
}
```

---

## Line Height

```
``css id="x6c8v2" p { line-height: 1.6; }
```

---

### # Lesson 8: CSS Box Model

Every HTML element can be considered as a box.

The CSS box model consists of:

1. Content
2. Padding
3. Border
4. Margin

---



## ## Content

The actual content of the element.

```
```css id="8r2w6m"
div {
  width: 300px;
}
```

Padding

Padding creates space inside the element.

```
```css id="k4p9x1" div { padding: 20px; }
```

---

## ## Border

Border surrounds the padding and content.

```
```css id="z6m3q8"
div {
  border: 2px solid black;
}
```

Margin

Margin creates space outside the element.

```
```css id="b7c2n5" div { margin: 20px; }
```

---

## ## Box Sizing

A commonly used rule is:

```
```css id="p5x8k2"
* {
  box-sizing: border-box;
}
```

This makes width and height calculations easier to manage.

Lesson 9: Width and Height

CSS can control the dimensions of elements.

```
```css id="d2m7v4" .box { width: 300px; height: 200px; }
```

```

Maximum Width

```css id="x8q3k6"
.container {
  max-width: 1200px;
}
```

Minimum Height

```
```css id="v5n9r2" .container { min-height: 300px; }
```

```

Lesson 10: Display Property

The `display` property controls how an element participates in layout.

Block

```css id="q2x6m8"
div {
  display: block;
}
```

Block elements normally start on a new line.

Inline

```
```css id="n4k8p3" span { display: inline; }
```

```

Inline-Block
```

```
```css id="m7c1v9"
button {
  display: inline-block;
}
```

None

```
```css id="w3j6s2" .hidden { display: none; }
```

The element is removed from the layout.

---

### # Lesson 11: CSS Positioning

The `position` property controls how elements are positioned.

#### ## Static

Default positioning.

```
```css id="a5x7m2"
.box {
  position: static;
}
```

Relative

The element remains in the normal flow but can be moved relative to its original position.

```
```css id="f8k2q6" .box { position: relative; top: 20px; left: 10px; }
```

---

#### ## Absolute

The element is positioned relative to its nearest positioned ancestor.

```
```css id="j3n7v5"
.box {
  position: absolute;
  top: 0;
  right: 0;
}
```

Fixed

The element stays fixed relative to the viewport.

```
```css id="r6c9x1" .navbar { position: fixed; top: 0; }
```

---

### ## Sticky

The element behaves normally until a specified scroll position is reached.

```
```css id="p8m4k7"
header {
  position: sticky;
  top: 0;
}
```

Lesson 12: Flexbox

Flexbox is a CSS layout system designed for arranging elements in one dimension.

Basic Flexbox

```
```css id="s4j8n2" .container { display: flex; }
```

---

### ## Flex Direction

```
```css id="x7q3m9"
.container {
  display: flex;
  flex-direction: row;
}
```

Or:

```
```css id="k2v6p8" .container { display: flex; flex-direction: column; }
```

---

### ## Justify Content

Controls alignment along the main axis.

```
```css id="c5n8r1"
.container {
  display: flex;
  justify-content: center;
}
```

Common values:

- flex-start
- center
- flex-end
- space-between
- space-around
- space-evenly

Align Items

Controls alignment along the cross axis.

```
```css id="y9m3q6" .container { display: flex; align-items: center; }
```

---

### ## Gap

Creates space between flex items.

```
```css id="v4k7x2"
.container {
  display: flex;
  gap: 20px;
}
```

Lesson 13: CSS Grid

CSS Grid is a two-dimensional layout system.

Basic Grid

```
```css id="n6p2w8" .container { display: grid; grid-template-columns: 1fr 1fr 1fr; }
```

This creates three equal columns.

---

## Grid Gap

```
```css id="m3x8q5"
.container {
  display: grid;
  gap: 20px;
}
```

Example

```
```css id="f7k4c2" .container { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; }
```

---

# Lesson 14: Pseudo-Classes

Pseudo-classes define special states of elements.

## Hover

```
```css id="8q2m6v"
button:hover {
  background-color: black;
  color: white;
}
```

This applies when the user moves the mouse over the button.

Focus

```
```css id="r5x9k3" input:focus { border: 2px solid blue; }
```

---

## First Child

```
```css id="j7p3n8"
li:first-child {
```

```
    color: red;
}
```

Lesson 15: Pseudo-Elements

Pseudo-elements style specific parts of an element.

Before

```
```css id="c6m2w9" h2::before { content: "★ "; }
```

```

After

```css id="x4k8q1"
h2::after {
  content: " ✓";
}
```

Lesson 16: CSS Transitions

Transitions make changes happen smoothly.

```
```css id="v9n3m7" button { background-color: blue; transition: 0.3s; }
```

```
button:hover { background-color: green; }
```

The color changes smoothly when the user hovers over the button.

```

Lesson 17: CSS Transforms

Transforms can move, rotate, scale, or skew elements.

Scale

```css id="p2x6k8"
.box:hover {
  transform: scale(1.1);
}
```

Rotate

```
``css id="m7q4c9" .box:hover { transform: rotate(45deg); }
```

```
---  
  
## Translate  
  
``css id="n5k8v2"  
.box {  
    transform: translateX(20px);  
}
```

Lesson 18: CSS Animations

CSS animations allow elements to change styles over time.

Creating an Animation

```
``css id="r3j7x5" @keyframes moveBox {
```

```
    from {  
        transform: translateX(0);  
    }  
  
    to {  
        transform: translateX(200px);  
    }  
}
```

```
}
```

Apply the animation:

```
``css id="k8m2p6"  
.box {  
    animation: moveBox 2s infinite;  
}
```

Lesson 19: Responsive Design

Responsive design allows websites to work properly on different screen sizes.

A responsive website should work on:

- Desktop
- Laptop
- Tablet
- Mobile phone

Media Queries

Media queries allow CSS rules to change based on screen size.

```
```css id="x6q9n3" @media (max-width: 768px) {
```

```
.container {
 flex-direction: column;
}
```

```
}
```

When the screen width is 768 pixels or smaller, the container changes to a column layout.

---

### ## Responsive Images

```
```css id="j4v7c2"  
img {  
  max-width: 100%;  
  height: auto;  
}
```

This prevents images from overflowing their containers.

Lesson 20: CSS Units

CSS supports different units.

Pixels

```
```css id="p8x3m6" h1 { font-size: 24px; }
```

---

## ## Percentage

```
```css id="v2k7q9"
.container {
  width: 80%;
}
```

Relative Units

Common relative units include:

- em
- rem
- %
- vw
- vh

Example:

```
```css id="c5n8x2" h1 { font-size: 2rem; }
```

---

## # Lesson 21: CSS Variables

CSS variables allow reusable values to be defined.

```
```css id="m7q2v6"
:root {
  --primary-color: blue;
  --main-spacing: 20px;
}
```

Use the variable:

```
```css id="x4k9p1" button { background-color: var(--primary-color); padding: var(--main-spacing); }
```

CSS variables make styles easier to maintain.

---

## # Lesson 22: CSS Specificity

When multiple CSS rules apply to the same element, CSS determines which rule has higher priority.

Generally, specificity follows this order:

1. Inline styles
2. ID selectors
3. Class selectors, attributes, and pseudo-classes
4. Element selectors

Example:

```
``css id="n3v8q5"
p {
 color: blue;
}

.text {
 color: green;
}

#message {
 color: red;
}
```

If the same paragraph has both the class and ID, the ID rule generally has higher specificity.

---

## Lesson 23: CSS Best Practices

Good CSS should be:

- Organized
- Readable
- Reusable
- Responsive
- Easy to maintain

### Use External CSS

Prefer:

```
``html id="r8m2x6" <link rel="stylesheet" href="style.css">
```

Instead of writing large amounts of inline CSS.

---

## ## Use Meaningful Class Names

Good:

```
```html id="k4p7n9"
<div class="student-card">
```

Less descriptive:

```
```html id="w2x6m3" <div class="box1">
```

---

## ## Avoid Unnecessary Repetition

Instead of:

```
```css id="q7v3m8"
.title {
  color: blue;
}

.subtitle {
  color: blue;
}
```

You can group them:

```
```css id="f5n8x2" .title, .subtitle { color: blue; }
```

---

## # Mini Project

### # Responsive Personal Portfolio

Create a responsive personal portfolio website using HTML and CSS.

## ## Requirements

The website should contain:

1. Header
2. Navigation bar
3. Hero section
4. About section
5. Skills section
6. Projects section

- 7. Contact section
- 8. Footer

---

## ## CSS Requirements

Use:

- External CSS
- Colors
- Typography
- Box model
- Flexbox
- CSS Grid
- Hover effects
- Transitions
- Media queries
- Responsive images
- CSS variables

---

## ## Suggested Structure

```
``text
portfolio/
|
├─ index.html
|
└─ style.css
```

---

# Practice Exercises

## Exercise 1: Styling Text

Create an HTML page and use CSS to:

- Change heading color.
- Change paragraph font size.
- Center a heading.
- Change the font family.

## Exercise 2: Create a Card

Create a student card containing:

- Student image
- Student name
- Course
- Description
- Button

Style the card using:

- Padding
  - Margin
  - Border
  - Border radius
  - Box shadow
- 

## Exercise 3: Flexbox Navigation

Create a navigation bar using Flexbox.

The navigation should contain:

- Home
- About
- Courses
- Contact

Arrange the links horizontally and add spacing between them.

---

## Exercise 4: CSS Grid

Create a three-column course layout.

Each course card should contain:

- Course title
- Description
- Duration
- Button

Use CSS Grid to arrange the cards.

---

## Exercise 5: Responsive Design

Make the course layout responsive.

On desktop:

Course 1 | Course 2 | Course 3

On mobile:

Course 1

Course 2

Course 3

Use a media query to achieve this.

---

## Final Quiz

1. What does CSS stand for?
  2. What is the purpose of CSS?
  3. What is the difference between inline, internal, and external CSS?
  4. What is a CSS selector?
  5. What is the difference between a class and an ID?
  6. What is the CSS box model?
  7. What is the difference between margin and padding?
  8. What is Flexbox?
  9. What is CSS Grid?
  10. What is the purpose of `justify-content`?
  11. What is the purpose of `align-items`?
  12. What is a media query?
  13. What is responsive design?
  14. What are pseudo-classes?
  15. What are pseudo-elements?
  16. What is a CSS transition?
  17. What is a CSS animation?
  18. What are CSS variables?
  19. What is CSS specificity?
  20. Why should external CSS generally be preferred for larger projects?
-

# Course Summary

After completing CSS Fundamentals, students should be able to style HTML webpages and create responsive layouts.

The major concepts covered are:

- Introduction to CSS
- Adding CSS
- CSS syntax
- Selectors
- Colors
- Backgrounds
- Text and fonts
- Box model
- Width and height
- Display property
- Positioning
- Flexbox
- CSS Grid
- Pseudo-classes
- Pseudo-elements
- Transitions
- Transforms
- Animations
- Responsive design
- CSS units
- CSS variables
- Specificity
- CSS best practices

## Next Step

After completing HTML and CSS, students should learn **JavaScript** to add logic, interactivity, and dynamic behavior to their webpages.